

Availability

Interview-focused learning material built on top of the source lesson: mental models, the nines table, real-world examples, trade-offs, misconceptions, interview questions and model answers, follow-up drills, practical exercises, and a senior-engineer perspective.

Beginner → Senior

India · UAE · Saudi

SDE-2 / SDE-3 / Tech Lead

Contents

1. Topic Overview
2. Simple Mental Model
3. Key Points to Remember
4. Real-World Examples
5. When to Use / When NOT to Use
6. Trade-offs
7. Common Misconceptions
8. Interview Questions (India · UAE · Saudi)
9. Interview Answers
10. Follow-up Questions
11. Practical Design Exercises
12. Quick Interview Revision Sheet
13. Senior Engineer Perspective

1. Topic Overview

Availability is a measure of how accessible and reliable a system stays for its users, expressed as uptime: the share of time the system is actually usable against the time it is promised to be usable. It exists because every real system fails sometimes — hardware dies, networks partition, deployments go wrong, traffic spikes beyond capacity — and availability is the discipline of designing so that individual failures don't become user-visible outages.

The problem availability solves is trust: users and downstream systems need to know a service will respond when called, within a promised window, most of the time. A system can be functionally correct and fast, and still be a failure in production if it is frequently unreachable.

Availability matters in System Design because it is usually the metric a business actually pays for — SLAs, customer trust, and revenue are tied to uptime more directly than to almost any other quality attribute. It also sits in tension with other properties covered elsewhere in this course: chasing availability means adding copies of things (redundancy, replication, geographic distribution), and the moment there is more than one copy of data, you inherit a consistency problem.

In a larger architecture, availability decisions shape almost everything downstream: how many regions you run in, whether your data store is single-primary or multi-primary, how aggressive your health checks and failover are, and how much engineering and operational cost you're willing to spend chasing the next nine.

Beginner framing

Availability = the system doesn't go down.

Senior framing

Availability = a measured, promised uptime for a specific, named part of the system — achieved by deliberately creating copies of things, which then forces an explicit consistency trade-off.

2. Simple Mental Model

The analogy: a chain of convenience stores instead of one supermarket

Imagine a city served by a chain of small convenience stores instead of one giant supermarket. If one store loses power or sells out of milk, customers just walk to another branch down the street — that's **redundancy**. The chain deliberately builds more stores than the bare minimum, and spreads them across different neighborhoods so one local blackout doesn't take out the whole city — that's **geographic distribution**. A sign (or an app) points customers to whichever nearby store is open and not too crowded — that's **load balancing**. Store managers check stock and equipment on a schedule, and head office gets an alert if a fridge stops working — that's **health monitoring and maintenance**.

The interesting question is the inventory app. When Store A sells the last carton of milk, does the app instantly show "0 left" everywhere (it has to phone every other store's till and wait for confirmation before updating — **strong consistency**, and the app is briefly slower or unavailable while it waits), or does it update Store A's count immediately and let the citywide total catch up a few seconds later (**eventual consistency**, so the app stays fast and available, but for a moment a customer elsewhere might be told "3 left" when there are really 2)?

Beginner understanding

More servers/locations = more available. Add copies and you're safer.

Interview-level understanding

Copies only help if they fail independently (different racks, zones, regions, power). And every copy you add is a new place data can disagree — so "how available" is never a free question; it always comes bundled with "how consistent, and measured against what promise."

3. Key Points to Remember ☆

- ★ **Definition:** Availability = uptime = operational time ÷ time the system is supposed to be operational.
- ★ **The seven strategies group into three buckets:** add copies (redundancy/replication, distributed storage, geographic distribution), spread the work (load balancing, consistency model), catch it early (health monitoring, regular maintenance).
- ★ **Every "add copies" strategy forces a consistency decision.** That's the throughline of this lesson: redundancy is not free, its price is paid in consistency.
- ★ **Redundancy without failure-domain diversity is fake redundancy.** Two "backup" servers in the same rack, same availability zone, or on the same power circuit can fail together.
- ★ **The CAP theorem is the formal version of "consistency costs availability":** during a network partition, a distributed system must choose between staying fully consistent or staying fully available — it cannot do both.
- ★ **Availability is always measured against a specific promise on a specific component** — a raw percentage with no scope stated is not a real answer in an interview.
- ★ **SLI / SLO / SLA are different things:** the SLI is what you actually measure, the SLO is your internal target, the SLA is the external, often contractual, promise (frequently with penalties for missing it).
- ★ **"Nines" of availability translate to concrete downtime budgets per year** — see the table in Section 12. Going from three nines to five nines is a large jump in engineering and operational cost for a shrinking amount of downtime.
- ★ **Consistency is a per-feature decision, not a global setting.** A payment ledger and a "likes" counter in the same product can — and usually should — use different consistency levels.
- ★ **Commonly forgotten:** maintenance and monitoring are availability strategies too, not just architecture. An unpatched, unmonitored redundant system still goes down.

4. Real-World Examples

These illustrate well-known, publicly documented patterns — not confirmed claims about any company's current non-public internals.

Databases with tunable consistency

Cassandra and **DynamoDB** let you choose a consistency level per read or write (e.g. ONE, QUORUM, ALL in Cassandra) — this makes the strong/weak/eventual choice from this lesson a literal, per-query dial rather than an abstract idea. **MongoDB** replica sets have a similar knob through read preference and write concern settings, trading latency and availability against how many replicas must confirm a write.

Cloud infrastructure

Cloud object stores commonly quote very high *durability* figures (the data won't be lost) that are a different metric from *availability* (the data can be reached right now) — a classic interview distinction. **Multi-AZ** and **multi-region** deployment patterns on **AWS**, **GCP**, and **Azure** are the direct implementation of "geographic distribution," and managed health checks plus auto-replacement of unhealthy instances are the direct implementation of "health monitoring."

Products and companies

Netflix is widely known for deliberately injecting failures into its own production systems (chaos engineering) specifically to verify that redundancy assumptions still hold — a direct response to the "redundancy without diversity is fake redundancy" risk. Messaging platforms like **WhatsApp** favor eventual consistency and retries for message delivery, because staying available and delivering slightly out-of-order is better than blocking. Ride-hailing and delivery platforms — **Uber**, **Careem**, **Talabat**, **Zomato** — typically treat driver-location updates as tolerant of a few seconds of staleness (eventual consistency, favoring availability), while treating fare or payment finalization with much stronger guarantees. Payment and fintech platforms — **Razorpay**, **PhonePe**, **PayTM**, **Tabby**, **Tamara** — are the standard example of preferring strong consistency on ledger/balance operations even when it costs some availability, because an available-but-wrong balance is worse than a temporarily unavailable one.

Caution

Treat the company examples above as illustrations of well-known industry patterns, not verified statements about current non-public architecture.

5. When to Use / When NOT to Use

When to Invest Heavily in Availability

- User-facing, revenue- or safety-critical systems, where downtime is direct financial or reputational loss (checkout, dispatch, payments).
- A global user base across time zones, where there is no safe "off hours" maintenance window.
- Read-heavy workloads that tolerate staleness and benefit from eventual consistency plus wide redundancy (feeds, catalogs, counters).

When Extreme Availability Is Overkill

- Internal tools, batch/analytics pipelines, admin dashboards — a maintenance window is fine, and five-nines infrastructure here wastes budget.
- Early-stage or low-traffic products — active-active multi-region is expensive and operationally heavy before you have the traffic or team to justify it.
- Systems where correctness must trump uptime — sometimes the right design is to fail closed (reject the request) rather than serve possibly-wrong data.

When to Lean Toward Strong Consistency

- Financial transactions, account balances, and anything gating a purchase or payment.
- Inventory counts or seat/ticket allocation where overselling causes real harm.
- Any data where "wrong but available" is worse than "unavailable for a moment."

When to Lean Toward Eventual / Weak Consistency

- Social feeds, like/view counters, recommendation data, presence indicators.
- Search index freshness, caching layers, analytics dashboards.
- Anywhere a few seconds of staleness causes no real user harm.

It depends on the requirements

There is no universal answer. Availability targets and consistency levels should be set per component, based on what happens to the business and the user if that specific piece of data is briefly wrong or briefly unreachable.

6. Trade-offs

Option	Advantages	Disadvantages	Best Use Case
Strong Consistency	Every reader sees the same, correct data; simplest mental model for correctness	Higher latency; availability suffers during network partitions or slow replicas	Payments, balances, inventory/seat allocation
Weak Consistency	Best availability and performance; replicas never block each other	Readers can see stale or conflicting data with no convergence guarantee timeline	Caches, ephemeral counters, best-effort telemetry
Eventual Consistency	Good availability and performance, with a guarantee that data does converge	Brief windows of stale reads; needs care for "read your own write" cases	Social feeds, catalogs, location updates, search indexes

Dimension by dimension

Performance: strong consistency adds coordination latency (waiting for confirmations); weak/eventual consistency avoids that wait, at the cost of temporary staleness.

Cost: redundancy and geographic distribution multiply infrastructure cost roughly with the number of copies kept; the jump from three nines to five nines is disproportionately expensive in both infrastructure and engineering time.

Complexity & Operational Overhead: multi-region active-active systems need failover automation, health checks, replication monitoring, and runbooks — all ongoing operational work, not one-time architecture.

Development Effort: handling eventual consistency correctly in application code (idempotency, conflict resolution, read-your-own-writes) is real engineering work that's easy to underestimate.

User Experience: a slow-but-correct response and a fast-but-briefly-wrong response fail differently in front of a user — the right choice depends entirely on what the data is for.

7. Common Misconceptions ⚠️

"High availability means the system never goes down."

WHY IT'S WRONG/INCOMPLETE

Availability is a statistical measure against a promise, not a guarantee of zero downtime. Even "five nines" allows about 5 minutes of downtime a year.

CORRECT UNDERSTANDING

The goal is to keep downtime within an agreed budget, and to make failures short and contained when they do happen — not to claim they'll never happen.

INTERVIEW-LEVEL NUANCE

Talk about error budgets: a target like 99.9% availability implicitly allows a known amount of downtime, which teams can "spend" deliberately (e.g. on a risky deploy) rather than treating every minute of downtime as a crisis.

"Adding redundant servers automatically gives you high availability."

WHY IT'S WRONG/INCOMPLETE

If the redundant copies share a failure domain — same rack, same availability zone, same power feed, same region — a single event can take all of them down together.

CORRECT UNDERSTANDING

Real redundancy requires failure-domain diversity: different racks, zones, or regions, ideally different providers for the most critical pieces.

INTERVIEW-LEVEL NUANCE

Be ready to name the specific failure domain your redundancy protects against, and the one it doesn't.

"You can have full availability and full consistency at the same time if you just engineer hard enough."

WHY IT'S WRONG/INCOMPLETE

During an actual network partition, the CAP theorem says a distributed system must choose: answer with possibly-stale data (favoring availability) or refuse to answer until it can confirm consistency (favoring consistency). This isn't an engineering shortfall, it's a structural limit.

CORRECT UNDERSTANDING

The choice can be made per-feature and can favor one side most of the time while gracefully degrading during rare partitions — but it cannot be avoided entirely.

INTERVIEW-LEVEL NUANCE

Naming CAP explicitly, and stating which side a specific feature falls on, reads as more senior than a vague "we'll make it both highly available and consistent."

""Eventual consistency" means the data is often wrong."

WHY IT'S WRONG/INCOMPLETE

In well-designed systems the convergence window is typically milliseconds to a few seconds, not minutes or hours.

CORRECT UNDERSTANDING

"Eventual" describes the guarantee's timing (not synchronous, but bounded and convergent), not the system's overall reliability.

INTERVIEW-LEVEL NUANCE

Be able to describe how your system handles the "read-your-own-write" problem — a common follow-up once eventual consistency is mentioned.

"A higher availability number is always a better engineering goal."

WHY IT'S WRONG/INCOMPLETE

Every additional nine costs disproportionately more in infrastructure, redundancy, and operational discipline, for a shrinking absolute amount of downtime avoided.

CORRECT UNDERSTANDING

The right availability target is set by business impact, not by an abstract preference for bigger numbers — an internal admin tool rarely needs the same target as a payment API.

INTERVIEW-LEVEL NUANCE

Naming the cost curve of "nines" and asking "available for whom, measuring what" shows the same maturity as the source lesson's own interview tip.

8. Interview Questions

Region-relevant company names are noted where the scenario naturally fits that market's product companies — this reflects the kind of system those companies publicly operate, not a claim that a specific company asked this exact question.

L1 Beginner

COMMON INTERVIEW QUESTION

What is availability in system design, and how is it typically measured?

COMMON INTERVIEW QUESTION

What does "five nines" availability mean, and roughly how much downtime does it allow per year?

COMMON INTERVIEW QUESTION

What is a single point of failure, and how do you remove one?

FREQUENTLY REPORTED QUESTION

What is the difference between availability and reliability?

L2 Intermediate

FREQUENTLY REPORTED QUESTION · Amazon, Flipkart, Careem, Noon

How does load balancing improve availability?

COMMON INTERVIEW QUESTION

What is the difference between redundancy and replication?

INTERVIEW-STYLE QUESTION

How would you design health checks for a fleet of servers, and what should happen when one fails a check?

INTERVIEW-STYLE QUESTION

Why might two "redundant" servers still go down at the same time?

L3 Senior

SENIOR-LEVEL FOLLOW-UP · fintech: Razorpay, PhonePe, PayTM, Tabby, Tamara

How does the CAP theorem relate to the trade-off between availability and consistency?

SENIOR-LEVEL FOLLOW-UP

How would you design a multi-region active-active architecture, and what does it cost you in consistency?

SENIOR-LEVEL FOLLOW-UP

When would you deliberately make part of a system less available in order to protect data correctness?

SENIOR-LEVEL FOLLOW-UP

How do you decide which consistency level — strong, weak, or eventual — to use for a given piece of data?

L4 Scenario-Based

SCENARIO-BASED QUESTION

Your SLA promises 99.95% availability, but you keep missing it during a specific two-hour window each day. How do you investigate and fix it?

SCENARIO-BASED QUESTION

One of your three availability zones goes completely offline. Walk through what happens automatically, and what you'd check manually.

SCENARIO-BASED QUESTION

Users report an item shows "in stock" after it was already sold out in another region. How do you explain this, and what would you change?

SCENARIO-BASED QUESTION

You need to run a database schema migration on a system that must stay available. How do you do it without downtime?

SCENARIO-BASED QUESTION · Flipkart Big Billion Days, Noon sale, Talabat/HungerStation rush

During a major sale event, the data center serving that region goes down mid-traffic-spike. What's your failover plan?

9. Interview Answers ☆

Q: What does "five nines" mean, and how much downtime does it allow?

MODEL ANSWER

Five nines is 99.999% availability, which allows roughly 5 minutes and 15 seconds of downtime a year. I'd immediately follow up by asking what's being measured — whole-site availability and single-API availability are very different promises, and the target should be set per component based on business impact, not applied blindly everywhere.

Q: How does load balancing improve availability?

MODEL ANSWER

A load balancer with health checks stops sending traffic to instances that are failing, so a single instance's failure doesn't become a user-visible outage — traffic just shifts to the healthy ones. It also lets me take instances out of rotation for maintenance without downtime, which is what turns "redundancy" from a static backup into something that's actually being exercised continuously.

Q: How does the CAP theorem relate to this lesson's availability-consistency trade-off?

MODEL ANSWER

CAP theorem is the formal statement of exactly what this lesson describes: once you have multiple replicas (which every availability strategy creates), a network partition forces a choice between answering with possibly-stale data (availability) or refusing to answer until consistency is confirmed (consistency). I'd apply that choice per feature — strong consistency for a payment ledger, availability-favoring eventual consistency for a product catalog — rather than picking one mode for the whole system.

Requirement: payment balance must never be shown or acted on incorrectly

Decision: use strong consistency for the balance/ledger service

Reason: an available-but-wrong balance risks double-spend or overdraft

Trade-off: higher write latency, reduced availability during partitions

Alternative: eventual consistency with compensating transactions/reconciliation — rejected here due to financial risk

Q: Why might two "redundant" servers still fail together?

MODEL ANSWER

If they share a failure domain — same rack, same power circuit, same availability zone, or even the same region — a single event like a power outage or a regional network issue takes both down at once. I'd place redundant instances in different zones at minimum, and different regions for the most critical services, and I'd periodically test that failover actually works rather than assuming it does.

Q: One availability zone goes completely offline. What happens?

MODEL ANSWER

Health checks detect the unhealthy instances in that zone within seconds, and the load balancer stops routing traffic there; if the data layer is also multi-AZ, a replica in a healthy zone is promoted or already serving reads. Users in other zones should see no impact; users pinned to that zone see a brief blip during failover. Afterward I'd check replication lag on the promoted replica and confirm no writes were lost.

Q: How do you run a schema migration without downtime?

MODEL ANSWER

I'd use an expand-and-contract approach: add the new column/table alongside the old one, deploy code that writes to both, backfill historical data, switch reads to the new shape, and only then remove the old column — each step independently deployable and reversible, so the system stays available throughout rather than needing a single risky cutover.

10. Follow-up Questions 🔥

Chain 1 — Choosing redundancy

INTERVIEWER

"Why did you add redundant instances?"

INTERVIEWER

"How do you know the redundant copy will actually take over?"

INTERVIEWER

"What if both copies are in the same data center?"

INTERVIEWER

"How do you test that failover actually works?"

INTERVIEWER

"What's your recovery time when the primary fails?"

Chain 2 — Choosing a consistency level

INTERVIEWER

"Why did you choose eventual consistency here?"

INTERVIEWER

"What happens if a user reads stale data right after writing?"

INTERVIEWER

"How long is your convergence window?"

INTERVIEWER

"Which parts of the system can't tolerate that staleness?"

INTERVIEWER

"How would you fix read-your-own-writes for those parts?"

Chain 3 — Measuring availability

INTERVIEWER

"How exactly are you calculating your 99.95%?"

INTERVIEWER

"Does that number count planned maintenance?"

INTERVIEWER

"What if only one endpoint is down but the rest of the site is fine?"

INTERVIEWER

"How do you know your health checks aren't lying to you?"

11. Practical Design Exercise

Exercise 1 — Highly Available Redirect Service

Design a URL-shortener redirect service with a 99.99% availability target.

Identify: requirements, traffic, storage, architecture, bottlenecks, scaling and availability strategy, failure scenarios, and trade-offs — before reading further.

Expected Thinking Process

- Translate 99.99% into a concrete downtime budget (~52 minutes/year) to calibrate how much redundancy is actually justified.
- Identify the redirect path as the critical, must-stay-up path, separate from the (less critical) link-creation/ analytics path.
- Plan failure-domain diversity: multiple instances across multiple availability zones, behind a load balancer with health checks.
- Decide the data layer's consistency needs: redirects can tolerate a cache with brief staleness; this favors availability.
- Plan monitoring: track the SLI (successful redirects / total redirect attempts) directly, not proxy metrics.

Reference Solution

Stateless redirect servers across at least two availability zones behind a load balancer with aggressive health checks, so a zone failure removes itself from rotation within seconds. A cache holds the hot short-code → URL mappings with high availability and eventual consistency — a few seconds of staleness on a newly created link is an acceptable trade for keeping the redirect path fast and always up. The source-of-truth database runs with a replica in a second zone for failover; link creation (a much lower-volume, less latency-sensitive path) can tolerate stronger consistency and slightly more latency. Alerts fire on the actual SLI — successful redirect rate — not just on server CPU or memory.

Exercise 2 — Checkout That Must Stay Up but Never Oversell

An e-commerce checkout system must stay available during a flash sale, but must never oversell inventory. Design it.

Identify: requirements, traffic, storage, architecture, bottlenecks, availability/consistency strategy, failure scenarios, and trade-offs — before reading further.

Expected Thinking Process

- Recognize this system has two very different consistency needs living side by side: browsing/catalog data, and the stock-count/reservation itself.
- Separate the "must never be wrong" data (remaining stock) from the "fine to be slightly stale" data (product listings, reviews, recommendations).
- Consider a reservation pattern: briefly hold stock for a user completing checkout, rather than a single global lock.
- Plan for the high-traffic, read-heavy browsing path to stay available and fast, independent of the stricter checkout path.
- Think about what happens if the strong-consistency stock service itself becomes a bottleneck under flash-sale load.

Reference Solution

Split the system by consistency need: product browsing and catalog data are served from horizontally-scaled, cached, eventually-consistent read replicas, so the high-traffic path stays available under load. The stock-count itself is owned by a single authoritative service using strong consistency (or a quorum-based store) with a short-lived reservation pattern — when checkout starts, a unit of stock is reserved for a few minutes rather than decremented and re-incremented repeatedly, which reduces contention while still preventing oversell. This keeps the vast majority of flash-sale traffic (browsing) highly available, while concentrating the strict consistency requirement — and its availability cost — on the smallest possible critical path.

12. Quick Interview Revision Sheet

30-Second Revision

Availability = uptime against a promise. Seven strategies: redundancy/replication, load balancing, distributed storage, consistency model, health monitoring, maintenance, geographic distribution. Every strategy adds copies, and copies force a consistency choice — strong (identical, costs availability), weak (may disagree), eventual (converges over time).

2-Minute Revision

- Availability = operational time ÷ time it should be operational, always measured against a specific, named promise.
- The seven strategies group into: add copies, spread the work, catch it early.
- Redundancy needs failure-domain diversity (different zones/regions) or it's not real redundancy.
- CAP theorem: during a network partition, choose availability or consistency — not both.
- Strong consistency: identical replicas, costs availability/latency. Weak: fastest, may disagree. Eventual: converges, balances both.
- SLI = what you measure, SLO = internal target, SLA = external promise (often contractual).
- Higher availability targets cost disproportionately more — set the target per component by business impact, not by default.

Interview Cheat Sheet — The Nines

Availability	Nines	Downtime / Year
99%	Two nines	~3.65 days
99.9%	Three nines	~8.76 hours
99.99%	Four nines	~52.6 minutes
99.999%	Five nines	~5.3 minutes
99.9999%	Six nines	~31.6 seconds

13. Senior Engineer Perspective

How a Senior Engineer Should Think About This

A senior engineer treats "what's our availability target" as an incomplete question until it's scoped: target for which component, measured how, against what promise. They resist the reflex to chase more nines everywhere, because each additional nine costs disproportionately more in infrastructure, redundancy, and operational discipline — and they can usually estimate that cost concretely enough to make the trade-off a real business conversation, not an abstract engineering preference.

They treat redundancy as something to verify, not assume — periodically testing failover (game days, chaos engineering) rather than trusting that a redundant setup drawn on an architecture diagram will actually behave that way under a real failure. They think in failure domains: which specific event (a rack losing power, a zone losing network, a region-wide provider incident) does this redundancy protect against, and which one would still take the whole system down.

They treat consistency as a per-feature decision embedded in the data model and API contract, not a single global setting — and they can articulate, for each critical piece of data, which side of the CAP trade-off it sits on and why. They use error budgets rather than treating every minute of downtime as equally bad: an agreed availability target implies an allowed amount of downtime, which can be spent deliberately (a risky but valuable deploy) rather than always treated as pure loss.

Observability is planned around the actual SLI users care about (successful checkout rate, successful redirect rate) rather than proxy signals like CPU or memory that can look healthy while the user-facing promise is already broken. And maintainability is judged by whether the failover and recovery procedures are documented and rehearsed well enough that any on-call engineer — not just the system's original author — can execute them at 3 a.m.

The senior-level default

Never present availability as "just add redundancy" or consistency as "always strong is safest." The correct senior answer is always: it depends on the requirements — and then show the reasoning: Requirement → Decision → Reason → Trade-off → Alternative.